SESSION 19

1.

Run a SELECT query on a large 'orders' table (at least 10,000 rows) to find all orders for a specific user_id and measure the query execution time.

```
SET @start_time = NOW(6);

SELECT *
FROM orders
WHERE user_id = 101;

SET @end_time = NOW(6);

SELECT
  TIMESTAMPDIFF(
    MICROSECOND,
    @start_time,
    @end_time
  ) / 1000 AS execution_time_ms;
```

Explanation:

1. NOW(6) records the start and end timestamps with microsecond precision.
2. SELECT * FROM orders WHERE user_id = 101 retrieves all orders belonging to user ID 101.
3. TIMESTAMPDIFF() calculates the elapsed time in milliseconds.
4. The orders table should contain at least 10,000 rows to meet the assignment requirement.

Result: The query displays all orders for user ID 101 and reports the elap

2.

**Create an index on the user_id column of the 'orders' table and re-run the same SELECT query to measure the new execution time.

Hint: Use CREATE INDEX idx_user_id ON orders(user_id); and compare the times before and after.**

```
CREATE INDEX idx_user_id

ON orders(user_id);
```

Run the SELECT Query Again:

```
SET @start_time = NOW(6);


SELECT *

FROM orders

WHERE user_id = 101;


SET @end_time = NOW(6);


SELECT

  TIMESTAMPDIFF(

    MICROSECOND,

    @start_time,

    @end_time

  ) / 1000 AS execution_time_ms;
```

Explanation:

1. CREATE INDEX creates an index named idx_user_id on the user_id column.
2. The SELECT query retrieves all orders belonging to user ID 101.
3. The start and end timestamps measure the elapsed time in milliseconds.
4. The index allows MySQL to locate matching rows more efficiently instead of scanning the entire table, depending on the query plan and data distribution.

Comparison of Execution Time:

Before Index	After Index
Record the execution time before creating the index.	Record the execution time after creating the index.
May require a full table scan.	May use the index to find matching rows.

Conclusion: Creating an index on user_id can improve query performance. Actual execution times depend on the table size, data distribution, caching, and database configuration.

3.

Use the EXPLAIN PLAN command to analyze how your SELECT query runs before and after adding the index, and write down the key differences you observe in the output.

```
EXPLAIN
SELECT *
FROM orders
WHERE user_id = 101;
```

SQL Query (After Creating the Index):

```
CREATE INDEX idx_user_id
ON orders(user_id);
```

```
EXPLAIN
SELECT *
FROM orders
WHERE user_id = 101;
```

Explanation:

The EXPLAIN command shows how MySQL plans to execute a query. By comparing the output before and after creating the index, we can identify changes in query performance.

Key Differences Observed:

Before Index	After Index
type may be ALL, indicating a full table scan.	type may be ref or range, indicating index-based access.
key is usually NULL.	key may show idx_user_id.
rows may show a large number of rows to examine.	rows may show fewer rows to examine.
Query may take more time.	Query may execute faster.

Conclusion:

Creating an index on user_id can reduce the number of rows MySQL examines and improve query performance. The actual output depends on the table's data and the query optimizer's decisions.

4.

Write a query for a 'products' table that avoids a full table scan by using an index on the 'category' column to fetch all products in a specific category.

```
CREATE INDEX idx_category
ON products(category);

SELECT *
FROM products
WHERE category = 'Electronics';
```

Explanation:

1. CREATE INDEX creates an index named idx_category on the category column.
2. WHERE category = 'Electronics' retrieves all products belonging to the Electronics category.
3. MySQL can use the index to locate matching products without scanning the entire table, depending on the query optimizer and data distribution.

Conclusion:

Creating an index on the category column can improve query performance by reducing the number of rows examined. Use EXPLAIN to verify whether MySQL uses the index instead of performing a full table scan.

5.

Suppose your SELECT query on the 'orders' table is still slow even after adding an index. Use EXPLAIN PLAN and research at least one more optimization technique (other than indexing) using an AI tool like ChatGPT or Copilot, and describe how you would apply it.

EXPLAIN

SELECT *

FROM orders

WHERE user_id = 101;

Explanation:

The EXPLAIN command helps analyze how MySQL executes a query. If the query is still slow after adding an index, we can check the number of rows examined, the access type, and whether the index is being used.

Optimization Technique: Avoid Using SELECT *

One optimization technique, other than indexing, is to select only the required columns instead of retrieving every column from the table.

Optimized SQL Query:

SELECT order_id, order_date, total_amount

FROM orders

WHERE user_id = 101;

How This Improves Performance:

1. It retrieves only the necessary columns.
2. It reduces the amount of data transferred from the database.
3. It can reduce memory usage and improve query response time, especially when the table contains many columns or large values.

Conclusion:

Using EXPLAIN helps identify query performance issues. Selecting only the required columns instead of using SELECT * is an effective optimization technique that can improve performance even when an index already exists.